STRATUM

Internal Knowledge Management Platform for Banking Teams

Architecture | RAG Pipeline | DevOps | MLOps | Security

v2.0 | FastAPI + React 19 + ChromaDB + Neon PostgreSQL + Groq LLM

"Knowledge that stacks."

API Version **v2.0**	Backend **FastAPI 0.115+**	Frontend **React 19 + Vite 7**
LLM **Groq llama-3.3-70b**	Embeddings **all-MiniLM-L6-v2**	Reranker **bge-reranker-base**
Vector DB **ChromaDB**	Primary DB **Neon PostgreSQL**	Auth **JWT HS256 + MFA**
CI/CD **GitHub Actions**	Containers **Docker + K8s**	Observability **Prometheus + Langfuse**

CONTENTS

1 Problem Statement and Solution

The Problem: Banking Knowledge Fragmentation

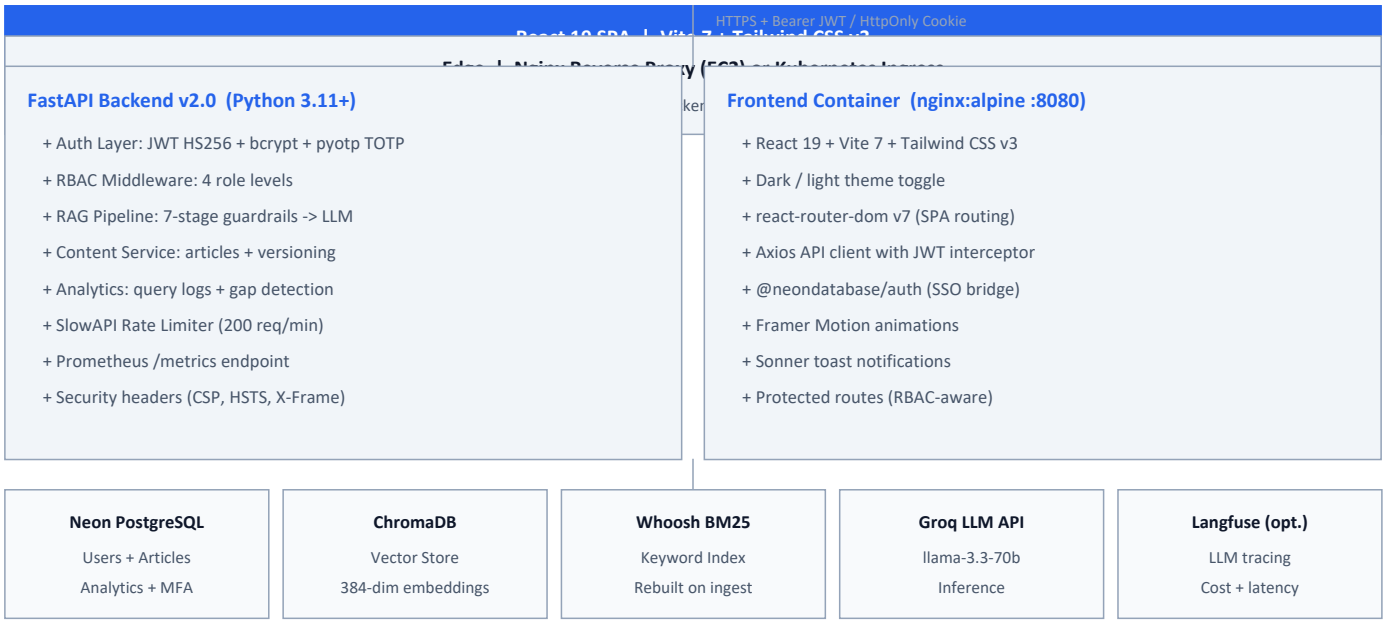
Challenge	Business Impact
Scattered documentation	Policies in SharePoint, runbooks in Confluence, diagrams in emails. No single source of truth.
High onboarding friction	New engineers spend 3-6 weeks learning institutional knowledge that should be findable in minutes.
Compliance risk	Outdated or misinterpreted policy documents create operational and regulatory exposure.
Poor search tools	Keyword-only search misses semantic context; staff cannot find what they do not know to search for.
No audit trail	No record of who accessed what knowledge, which questions go unanswered, or where content gaps exist.
Expert bottlenecks	SMEs become single points of failure because domain knowledge is not codified or searchable.

The Solution: What Stratum Delivers

Stratum Capability	How It Solves the Problem
Unified knowledge portal	All approved docs in one searchable, governed platform with lifecycle and version control.
Hybrid RAG AI copilot	Vector + BM25 + CrossEncoder reranking + Groq LLM. Every answer cites its source documents.
Article governance	Draft -> In Review -> Published -> Archived. Full version history per edit.
Document quality gate	PDF upload runs pdfplumber heuristics. Tier: ok / warn / fail before any indexing.
Knowledge gap analytics	Every unanswered query is logged. Admins see aggregated gaps to guide content creation.
RBAC and audit trail	Role-gated access, query logs, admin event log, Langfuse LLM tracing.
MFA and SSO	TOTP multi-factor auth + Neon Auth Google/email SSO in addition to email+password.

2 System Architecture

Stratum follows a layered architecture: React SPA -> Nginx Edge -> FastAPI Backend -> Data Stores + External Services. Each layer has a single responsibility and communicates via well-defined interfaces (HTTPS/JWT, SQLAlchemy, ChromaDB client, Groq REST API).



3 RAG Pipeline: Ingestion and Query Path

Document Ingestion Pipeline (LangGraph state machine)

Stage	What It Does
extract.py	pdfplumber: text + tables + page metadata. File-hash dedup skips unchanged files.
clean.py	Normalize whitespace, strip headers/footers, preserve page-level metadata.
chunk.py	tiktoken cl100k_base. 400-token windows. 80-token overlap. Table-aware splits.
embed.py	SentenceTransformers all-MiniLM-L6-v2. 384-dim vectors. Batch size 64.
store.py	Write chunks + embeddings to ChromaDB. Rebuild Whoosh BM25 index from Chroma docs.

Real-time Query Pipeline (per request; singletons loaded once per process)

Stage	Detail
1 Input Guardrail	Injection regex + blocklist check. Blocked -> HTTP 400 before any retrieval.
2 Hybrid Retrieval	Chroma dense search (cosine, top-K) + Whoosh BM25 (TF-IDF, top-K). Run in parallel.
3 RRF Fusion	Reciprocal Rank Fusion merges both ranked lists. Score = 1/(k+rank), k=60. No calibration needed.
4 CrossEncoder Rerank	BAAI/bge-reranker-base scores each (query, chunk) pair. Chunks below threshold are dropped.
5 Output Guardrail	PII masking (phone, email, card regex). If no chunk exceeds threshold -> 'not enough info'.
6 Groq LLM Generation	llama-3.3-70b-versatile. Banking system prompt + anti-hallucination instruction. Temperature 0.
7 Citations + Logging	Source document names and page numbers extracted. Query logged: answered, latency_ms, source_ids.

Why Hybrid Retrieval?

Signal	Handles	Component
Semantic (vector)	Conceptual and paraphrased queries	ChromaDB cosine similarity
Keyword (BM25)	Exact regulatory codes: KYC, AML, PSD2, FATF, SWIFT	Whoosh BM25
RRF Fusion	Combines both ranked lists without score normalization	RRF k=60
CrossEncoder	Accurate pairwise relevance; applied to shortlist only	bge-reranker-base

Guardrails Design

Layer	Triggers On	Action
Input: injection check	Override patterns ('ignore instructions', 'system:', etc.)	HTTP 400 - blocked before RAG
Input: blocklist	Out-of-scope topic categories	HTTP 400 - not processed
Output: low confidence	No chunk above RERANKER_THRESHOLD	Return 'insufficient info' - no LLM call
Output: PII masking	Phone, email, 16-digit card numbers in LLM output	Regex strip post-generation

Configuration Variables

Variable	Default	Description
EMBEDDING_MODEL	all-MiniLM-L6-v2	HuggingFace embedding model name
RERANKER_MODEL	BAAI/bge-reranker-base	CrossEncoder reranker model
GROQ_MODEL	llama-3.3-70b-versatile	Groq LLM model ID
TOP_K_VECTOR	20	Dense neighbours to retrieve
TOP_K_BM25	20	BM25 matches to retrieve
RERANKER_THRESHOLD	0.0	Minimum CrossEncoder score to keep chunk
TOP_K_FINAL	5	Max chunks passed to LLM after reranking
CHUNK_SIZE	400 tokens	tiktoken cl100k_base window size
CHUNK_OVERLAP	80 tokens	Overlap between consecutive chunks

4 Authentication and Session Model

Auth Path	Endpoint	Mechanism
Email / Password	POST /auth/login	bcrypt hash verification. JWT access token (15 min) + HttpOnly refresh cookie (7 days). MFA challenge if TOTP enabled.
Google / Neon SSO	POST /auth/neon/exchange	User completes Neon Auth OAuth in browser. SPA sends Neon token; API verifies and issues Stratum JWT. User upserted with neon_auth_sub.
MFA Setup	POST /auth/mfa/setup+confirm	pyotp TOTP. QR code URI returned for authenticator app. Secret stored in users table.
MFA Login	POST /auth/mfa/verify-login	TOTP 6-digit code or single-use backup code (bcrypt-hashed). On success, full token pair
Token Refresh	POST /auth/refresh	HttpOnly cookie sent automatically. Returns new access token. Refresh hash validated
Logout	POST /auth/logout	Clears refresh hash in DB. All active sessions invalidated. Cookie cleared.

Token Security Model

Token	Properties	Security Note
Access token	JWT HS256, 15-min expiry, role + user_id claims	Stored in memory by SPA; never in localStorage
Refresh token	HttpOnly cookie, 7-day expiry	JS cannot read it; safe from XSS. Sent automatically on
Refresh hash in DB	SHA-256 hash stored on users row	Logout and password changes immediately invalidate all sessions globally.
MFA backup codes	bcrypt hashed, limited count, single-use	User must regenerate after exhaustion. Cannot be viewed again after creation.

5 Article Lifecycle and Knowledge Governance

State	Who Can Transition	Visibility
Draft	Author creates; author + admins edit	Only author and admins
In Review	Author submits; knowledge_admin/expert reviews	Author and admins
Published	knowledge_admin or domain_expert approves	All employees; appears in search and RAG
Archived	knowledge_admin or system_admin archives	Hidden from search; version history retained

Each article edit creates a snapshot in article_versions (author_id, timestamp, full body). Status transitions are recorded in the audit event log accessible to system_admin.

6 RBAC: Roles and Portal Access

Role	Knowledge Hub	KB Management	Admin Console	User Mgmt
employee	Search, Chat, Ask, Bookmarks, Feedback, Learning paths	Read published only	No access	No access
knowledge_admin	Full access	Create, Edit, Publish, Upload PDFs	Analytics, Gaps, Feedback	No access
domain_expert	Full access	Create, Edit, Review articles	Analytics, Gaps	No access
system_admin	Full access	Full KB management	Full admin console	Users CRUD + Audit lo

7 DevOps and CI/CD Pipeline

Full lifecycle: developer push -> GitHub Actions CI -> Docker build + Trivy scan -> Tag v*.*.* -> image push to GHCR -> SSH deploy to EC2 -> smoke test -> auto-rollback on failure.

CI Checks (every push/PR)

Tool	Type	What It Checks
gitleaks	Secret scanning	Blocks any credential or API key committed to repo
ruff	Python linting + style	Applied to app/ + ingestion/ + tests/
mypy	Python type checking	Strict annotations enforced on app/
bandit	Python SAST security	OWASP-aligned; flags common Python security issues
pytest	Unit + integration tests	Postgres 16 service container; tests in backend/tests/
ESLint	JavaScript/React lint	eslint 9 flat config
Vite build	Frontend prod build smoke	Validates JSX compilation and import resolution
Trivy	Container CVE scan	CRITICAL severity exits 1, blocks merge on main branch

CD: Release Process (tag v*.*.*)

Step	Where	Action
1. Tag	Local	git tag v1.2.0 && git push origin v1.2.0
2. Build	GitHub Actions	docker build backend + frontend; inject VITE_* build-time env from GH secrets
3. Push	GitHub Actions	docker push to GHCR: ghcr.io/org/stratum-backend:v1.2.0
4. SSH	GitHub Actions	SSH to EC2 host; run deploy/docker/scripts/deploy.sh
5. Deploy	EC2 host	docker compose pull + docker compose up -d --remove-orphans
6. Smoke	EC2 host	GET /api/v1/ping -> assert HTTP 200 + {status:ok}
7. Done or rollback	EC2	Rollback: export APP_TAG_PREVIOUS=v1.1.0; run rollback.sh

Docker Compose Services (deploy/docker/docker-compose.yml)

Service	Image	Port	Purpose
backend	ghcr.io/org/stratum-backend:${APP_TAG}	8000	FastAPI + Uvicorn
frontend	ghcr.io/org/stratum-frontend:${APP_TAG}	8080	nginx serving React SPA
redis	redis:7-alpine	6379	Rate limit storage (internal)
prometheus	prom/prometheus	9090	Metrics (observability profile)
grafana	grafana/grafana	3000	Dashboards (observability profile)

Observability Stack

Tool	Interface	Captures
Prometheus	/metrics (auto-instrumented)	HTTP request rate, error rate, latency histogram per endpoint
Grafana	Dashboard at :3000	Prometheus visualizations; alert on 5xx spike, high P95 latency
Langfuse	Cloud SaaS (optional)	LLM traces: prompt, output, token count, cost, end-to-end latency
Query logs	Postgres query_logs table	Every RAG query: user, text, answered flag, latency_ms, source_ids
Feedback	Postgres feedback table	Thumbs up/down + optional comment per answer via POST /feedback

8 MLOps Lifecycle

Stratum's ML surface is retrieval + LLM inference, not a custom training loop. MLOps focuses on corpus versioning, ingestion pipeline management, evaluation, and deployment alignment.

Stage	Current Tooling	Recommended Extension
1. Data	PDF corpus in data/raw/. File-hash dedup on ingest.	DVC or LakeFS for regulatory reproducibility. S3 for corpus
2. Feature Eng.	LangGraph: extract->chunk->embed. 400t/80 overlap. all-MiniLM-L6-v2.	Evaluate semantic/heading-aware chunking for long policy
3. Evaluation	Offline: Ragas on eval_qa.json. Production: Langfuse traces. User: /feedback	Wire evaluate_rag.py to nightly CI; alert if faithfulness < 0.80.
4. Deployment	Same Docker image as API. Models as singletons. RAG_WARMUP_ON_STARTUP=true.	Changing EMBEDDING_MODEL requires full reindex - vector store and model must match.
5. Monitoring	Prometheus HTTP metrics. Grafana dashboards. Knowledge gap growth rate.	Alert: faithfulness < 0.80, P95 > 10s, feedback < 65% positive, >20 new gaps/week.

Ragas Evaluation Metrics

Metric	Target	Low Score Means	Action
faithfulness	> 0.85	LLM adding info not in context (hallucination)	Strengthen system prompt; reduce
answer_relevancy	> 0.80	Answer does not address the question	Check chunking quality; improve
context_recall	> 0.75	Relevant chunks not being retrieved	Increase TOP_K_VECTOR/BM25; check
context_precision	> 0.70	Retrieved chunks mostly irrelevant (noise)	Increase RERANKER_THRESHOLD; tune RRF k

Pipeline Gaps and Enhancement Roadmap

Gap	Priority	Impact	Suggested Fix
No query rewrite / HyDE	High	Vague queries hurt recall	LLM-based query expansion behind feature flag
Fixed chunk windows	Med	Long policies split at bad boundaries	Heading-aware or semantic sentence splits
No eval in CI	High	Faithfulness regression caught too late	Wire evaluate_rag.py to nightly CI job
Source diversity (MMR)	Med	Same PDF may fill all top-K slots	Post-rerank MMR or deduplicate by file_name
No streaming responses	Low	Full response blocks until Groq returns	SSE streaming from Groq to SPA

9 API Endpoints Reference

All endpoints under /api/v1. Swagger: http://localhost:8000/docs (disabled in production unless EXPOSE_API_DOCS=true).

Method	Path	Auth	Purpose
GET	/api/v1/ping	Public	Health check
POST	/api/v1/auth/login	Public	Email+password -> JWT + MFA step if enabled
POST	/api/v1/auth/neon/exchange	Public	Neon Auth token -> Stratum JWT
POST	/api/v1/auth/mfa/verify-login	Public	Complete MFA TOTP challenge
POST	/api/v1/auth/refresh	Cookie	Refresh access token via HttpOnly cookie
POST	/api/v1/auth/logout	Bearer	Invalidate refresh token + clear session
GET	/api/v1/auth/me	Bearer	Get current user profile + role
PUT	/api/v1/auth/me	Bearer	Update profile (name, bio, preferences)
POST	/api/v1/auth/me/avatar	Bearer	Upload avatar (disk or S3)
POST	/api/v1/auth/change-password	Bearer	Change own password
POST	/api/v1/auth/mfa/setup	Bearer	Begin TOTP enrollment; returns QR URI
POST	/api/v1/auth/mfa/confirm	Bearer	Confirm TOTP with first code
POST	/api/v1/auth/mfa/disable	Bearer	Disable MFA (requires current TOTP)
POST	/api/v1/chat	Employee	Multi-turn conversational RAG with history
POST	/api/v1/ask	Employee	Single-turn RAG Q&A with source citations
POST	/api/v1/search	Employee	Hybrid search - no LLM generation
POST	/api/v1/feedback	Employee	Submit thumbs up/down rating
GET	/api/v1/articles	Employee	List published articles (paginated)
GET	/api/v1/articles/{id}	Employee	Get article by ID
GET	/api/v1/articles/{id}/pdf	Employee	Download source PDF for article
GET/POST/D	/api/v1/bookmarks	Employee	Manage saved articles
POST	/api/v1/articles	KnowledgeAdmin	Create new article (draft)
PUT	/api/v1/articles/{id}	KnowledgeAdmin	Edit article content
DELETE	/api/v1/articles/{id}	KnowledgeAdmin	Delete article
POST	/api/v1/articles/{id}/submit-review	KnowledgeAdmin	Submit for review workflow
POST	/api/v1/articles/{id}/approve	KnowledgeAdmin	Approve and publish
POST	/api/v1/articles/{id}/archive	KnowledgeAdmin	Archive published article
POST	/api/v1/admin/upload-pdf	SourcesAdmin	Upload PDF -> quality check -> ingest
GET	/api/v1/admin/sources	SourcesAdmin	List all ingested PDF sources
DELETE	/api/v1/admin/sources/raw/{file}	SourcesAdmin	Remove source PDF file
POST	/api/v1/admin/reindex	SourcesAdmin	Trigger full LangGraph reindex
GET/POST	/api/v1/admin/users	SystemAdmin	List users / create new user
PATCH	/api/v1/admin/users/{id}	SystemAdmin	Update user role or status
DELETE	/api/v1/admin/users/{id}	SystemAdmin	Delete user (tombstoned)
GET	/api/v1/analytics/summary	Admin	Usage stats and top queries
GET	/api/v1/admin/knowledge-gaps	Admin	Unanswered query aggregation
GET	/api/v1/admin/query-logs	Admin	Full query history
GET	/api/v1/admin/audit-events	Admin	Security and admin event log
GET	/api/v1/admin/feedback	Admin	All user answer ratings
GET	/metrics	Internal	Prometheus metrics scrape

10 Full Tech Stack

Layer	Technology	Purpose
Backend API	FastAPI 0.115+ + Uvicorn	Async HTTP, OpenAPI docs, middleware chain
Python runtime	3.11+	Type-annotated; ruff/mypy enforced in CI
Primary DB	PostgreSQL via Neon (serverless)	Users, articles, analytics, MFA; pooled connections
Vector Store	ChromaDB (local persistent)	Cosine similarity; 384-dim embeddings
Keyword Index	Whoosh BM25	TF-IDF keyword index; rebuilt from Chroma on ingest
Embeddings	all-MiniLM-L6-v2 (SentenceTransformers)	384-dim dense vectors; CPU-friendly
Reranker	BAAI/bge-reranker-base (CrossEncoder)	Pairwise relevance scoring; max_length=512
LLM	Groq API - llama-3.3-70b-versatile	Grounded generation; banking system prompt
Ingestion	LangGraph pipeline	Typed state machine: extract->clean->chunk->embed->store
Observability	Prometheus + Grafana + Langfuse	HTTP metrics, dashboards, LLM traces
Auth	JWT HS256 + bcrypt + pyotp + Neon Auth	Password, MFA TOTP, Google SSO
Frontend	React 19 + Vite 7 + Tailwind CSS v3	SPA, dark mode, react-router-dom v7
Rate Limiting	SlowAPI + Redis	200 req/min per IP; Redis-backed in production
Containers	Docker + Docker Compose	Dev and production multi-service stacks
Orchestration	Kubernetes + Kustomize	K8s base + production overlay; HPA support
Infrastructure	Terraform	Namespace provisioning + kube-prometheus-stack Helm
CI/CD	GitHub Actions	Lint, test, Docker build, Trivy, deploy on tag
PDF extraction	pdfplumber	Text + table extraction; quality tier assessment
Eval framework	Ragas	Faithfulness, answer_relevancy, context metrics

11 Database Schema

Table	Description
users	Identity, RBAC role, profile fields, bcrypt password, TOTP secret, lockout, refresh hash, neon_auth_sub
deleted_users	Tombstone: blocks re-registration after admin delete
articles	KM content: title, body (markdown), status, author_id, source PDF references
article_versions	Full snapshot per edit: author, timestamp, body; retains history after status changes
bookmarks	User <-> article many-to-many; created_at timestamp
learning_paths	Curated content sequences: name, description, owner, visibility
learning_path_items	Articles/resources within a path: order, path_id, article_id
user_progress	Per-user progress through learning paths: completed items, last_accessed
expert_profiles	SME registry: domain, skills, contact, user_id FK
query_logs	Every RAG query: user_id, query_text, answer, answered bool, latency_ms, source_ids, created_at
feedback	User ratings: query_log_id FK, rating (+1/-1), comment, user_id, created_at

12 Security Posture

Control	Implementation
Transport	HTTPS everywhere. HSTS in production (max-age=31536000, includeSubDomains). Let's Encrypt / Certbot TLS.
Identity	JWT HS256 (15 min). HttpOnly refresh cookie (7 days). bcrypt passwords. pyotp TOTP MFA.
Authorization	RBAC dependency on every API route. 4 role levels. Portal routes gated in both API and SPA.

Control	Implementation
Input safety	Prompt injection guardrail + blocklist check before any RAG processing.
Output safety	PII regex masking on LLM output. Confidence threshold gate to prevent low-certainty hallucinations.
Security headers	X-Content-Type-Options: nosniff. X-Frame-Options: DENY. X-XSS-Protection. Referrer-Policy. Permissions-Policy. CSP default-src
Request limits	MAX_REQUEST_SIZE_MB (default 50 MB) enforced in middleware before route handlers run.
Rate limiting	SlowAPI 200 req/min per IP. Redis-backed in production (memory:// rejected in prod startup check).
Secrets management	No secrets in Git. K8s Secrets / env files on host only. gitleaks scans every CI push.
Container security	Trivy CRITICAL CVE scan on every main-branch Docker build. Blocks merge on critical findings.
Code security	bandit SAST on Python code. ruff linting. Both enforced in CI.
API docs	Swagger/ReDoc disabled in ENVIRONMENT=production unless EXPOSE_API_DOCS=true.

13 Quick Setup Reference

Minimum Required Secrets (backend/.env.local)

GROQ_API_KEY	gsk_... (from console.groq.com)

DATABASE_

Local Development (step by step)

- Step 1: pip install -r backend/requirements.txt (from backend/ directory)
- Step 2: npm install (from repo root; installs concurrently for npm run dev)
- Step 3: cp config/env/backend.env.local backend/.env.local then fill in the 3 required secrets above
- Step 4: cp config/env/frontend.vite.example frontend/.env and set VITE_API_BASE=/api/v1
- Step 5: npm run dev (from repo root; starts FastAPI :8000 and Vite :5173 together)
- Step 6: curl http://127.0.0.1:8000/api/v1/ping -> {"status":"ok","service":"stratum-api"}
- Step 7: Open http://localhost:5173 in the browser

Ingest Documents into the Knowledge Base

- Place PDF files in backend/data/raw/
- cd backend then python -m ingestion.main
- Pipeline: extract -> clean -> chunk (400t/80 overlap) -> embed -> ChromaDB + Whoosh rebuild
- Admin reindex also available: POST /api/v1/admin/reindex (requires ENABLE_ADMIN_REINDEX=true)

Docker Production Stack

- Create backend/.env from config/env/backend.env.prod (fill in all required secrets)
- docker compose -f deploy/docker/docker-compose.yml up -d --build
- Add --profile observability for Prometheus :9090 + Grafana :3000

Production Release via GitHub Actions

- Merge to main -> CI must be green (ruff, mypy, bandit, pytest, ESLint, Trivy)
- git tag v1.2.0 && git push origin v1.2.0 -> triggers deploy.yml automatically
- GitHub Actions: builds images -> pushes to GHCR -> SSH to EC2 -> deploy.sh -> smoke-test.sh
- Rollback: export APP_TAG_PREVIOUS=v1.1.0; run deploy/docker/scripts/rollback.sh

Documentation Map

Document	Contents
README.md	Setup, endpoints, schema, contributing guide
docs/HOW_IT_WORKS.md	Problem statement, product flows, RAG + auth + governance flows
docs/system-design.md	Architecture Mermaid diagrams (10 diagrams across all layers)
docs/RAG_PIPELINE.md	Pipeline audit, gaps, eval, configuration, enhancement roadmap
docs/DEVOPS.md	CI/CD pipeline, Docker, K8s, observability, security controls
mlops/README.md	MLOps lifecycle, Ragas eval, monitoring alerts, recommended extensions
docs/POSTGRES_DATABASE.md	Neon setup, schema, MFA model details
docs/AWS_EC2_PRODUCTION_GUIDE.md	EC2 bootstrap, TLS, deploy, rollback, backup/restore SLOs
config/SECRETS.md	Secret management patterns and full env var reference
deploy/k8s/README.md	Kubernetes Kustomize deployment guide
infra/terraform/README.md	Terraform namespace + kube-prometheus-stack bootstrap